

CINECA AGENTIC PLATFORM

An Enterprise-Grade Multi-Tenant AI Orchestrator with MCP Tools and Secure Graph Querying

University:

Sapienza University of Rome

Department:

Information Engineering, Electronics and Telecommunications

Academic Year:

2025/2026

University Advisor:

Prof. Maria De Marsico

Company:

CINECA

Author:

Arman Feili

University Co-Advisors:

Prof. Marco Raoul Marini

Dr. Valerio Venanzi

CINECA Advisors:

Dr. Giuseppe Melfi

Dr. Marco Puccini



SAPIENZA
UNIVERSITÀ DI ROMA

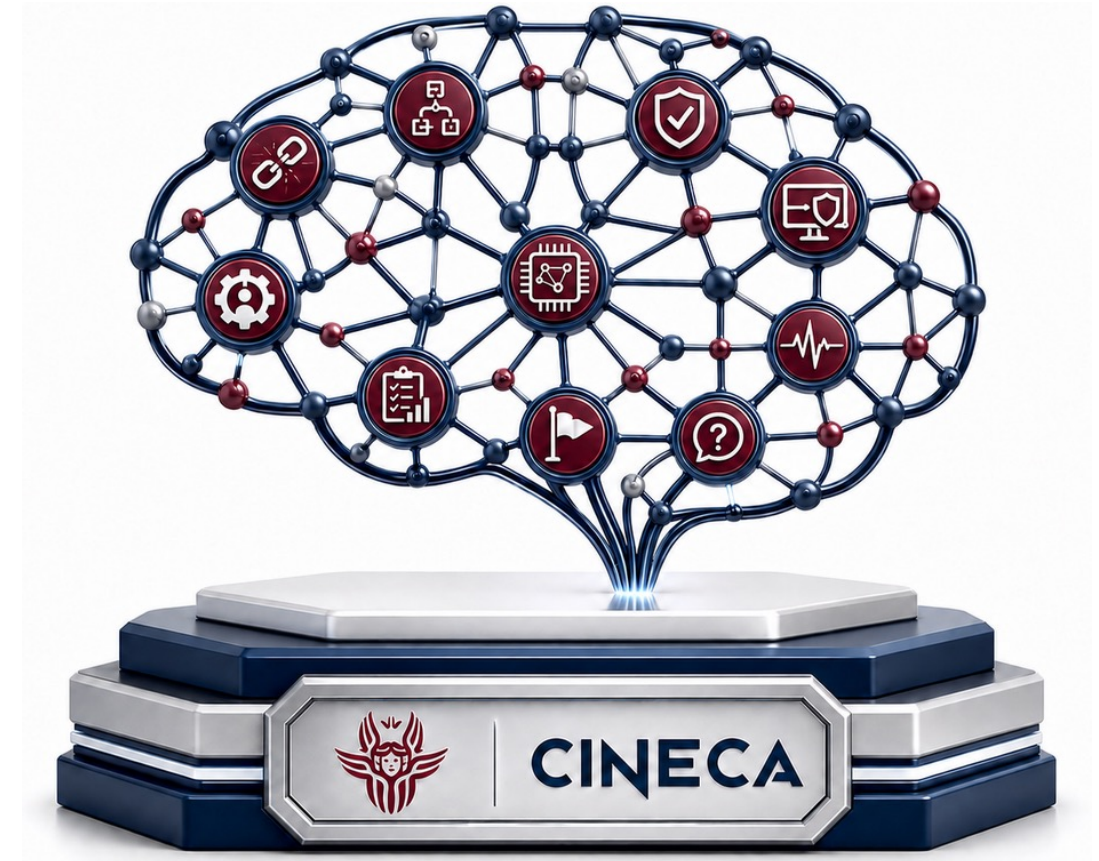
All rights relating to this teaching material and its contents are reserved by Sapienza and its authors (or teachers who produced it). Personal use of the same by the student for study purposes is permitted. Its dissemination, duplication, assignment, transmission, distribution to third parties or to the public is absolutely prohibited under penalty of the sanctions applicable by law.

CINECA

TABLE OF CONTENTS

Presentation structure

1. Production Gap
2. What Was Delivered
3. System Workflow & Platform Architecture
4. UI, Security & API Layer
5. Job Execution & Orchestrator
6. Validation, Results & Comparison
7. Conclusion & Future Work
8. Q&A



PROBLEM & MOTIVATION

Production gap

The problem

- Users need to **access** the Bioinformatic **graph data**, but they might not know how to write **Cypher** commands.
- They might **depend on developers** to access it.
- This creates **delays** and keeps data locked.
- There is a need for a **secured** and not too technical tool.

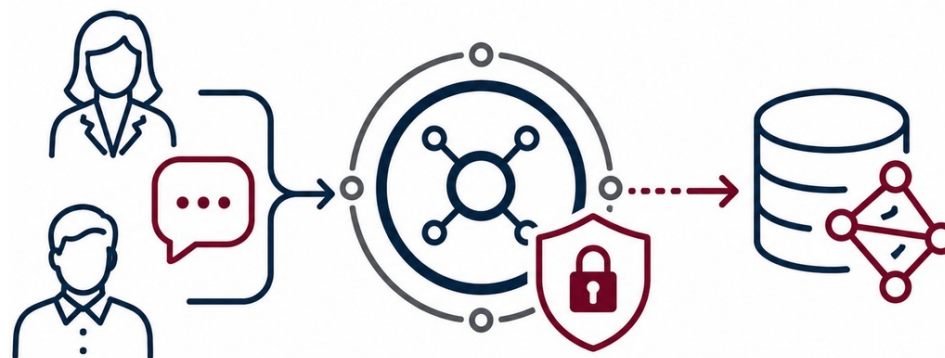
Solution

CINECA needs an **agentic** platform that turns natural-language requests into **secure, traceable, and reproducible** graph commands, while staying **scalable** and **flexible** across LLM providers.

Why an Agentic Platform?

Because simple chatbots lack:

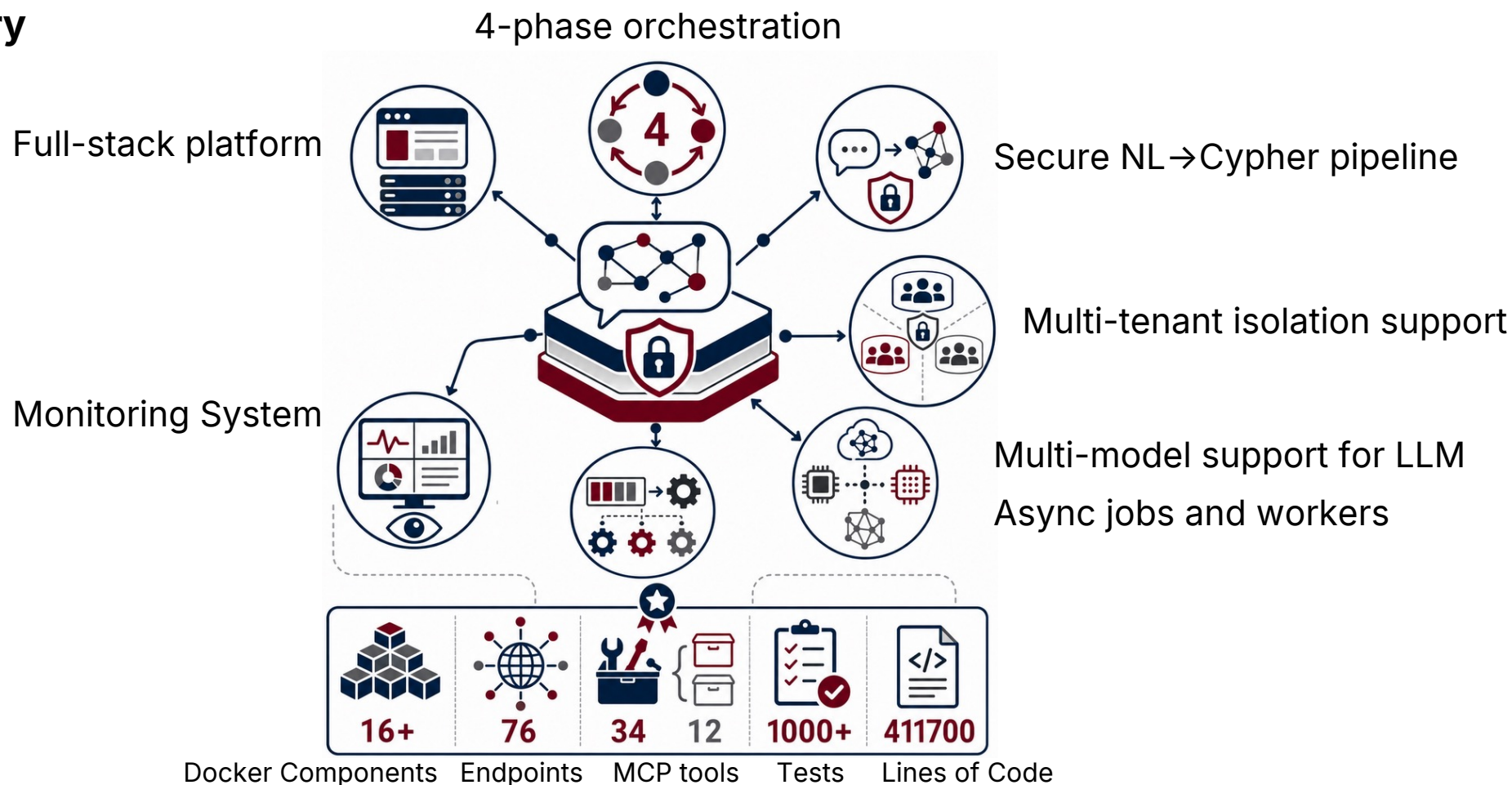
- **Multi-step** execution
- Secure database and **tool access**
- **RBAC** and **tenant** isolation



WHAT WAS DELIVERED

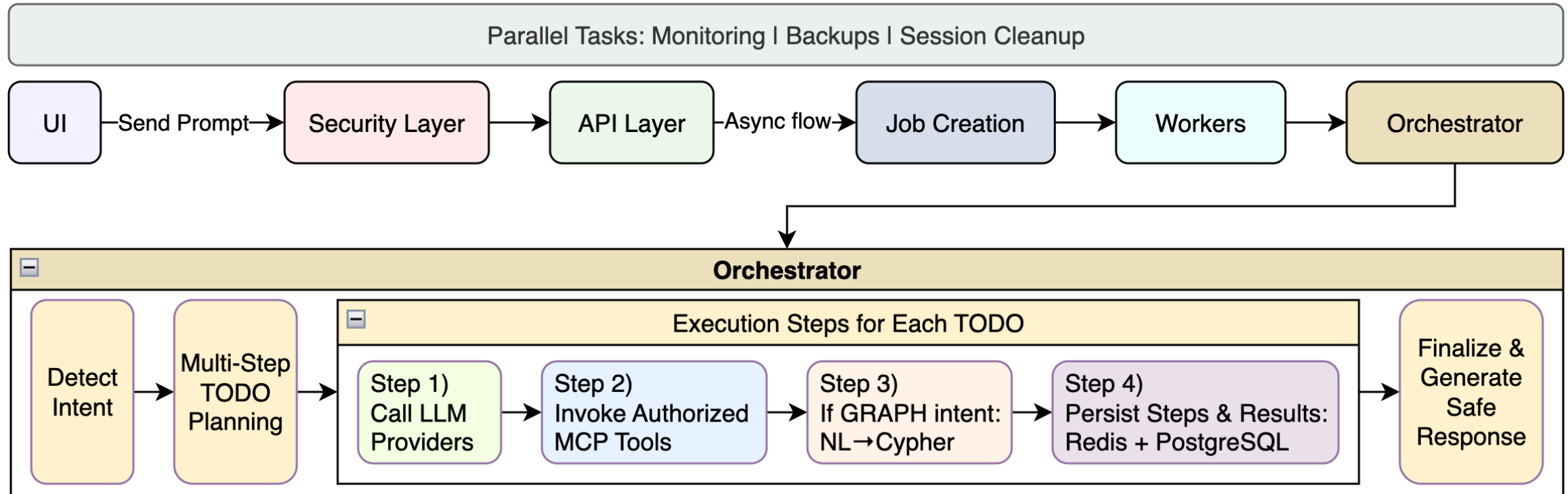
A production-ready agentic platform for secure natural-language access to graph data.

Core Delivery



SYSTEM WORKFLOW

Platform Architecture



UI

Admin & User chatbot UI

Main Flow

Users log in, choose model, send the prompt, see the live progress and the result.

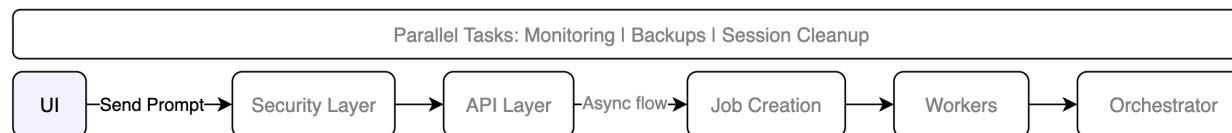
Agent Chat UI

- Built with **Next.js**
- Have a **text box** and **selected model**
- Shows **live progress** and **execution steps**
- Give **final result**

Identity & Access

- Uses **OAuth 2.0** as a 3rd party service for defining roles and scopes
- Does **JWT-based Authentication and Authorization**
- Control **Tenant, role, and permissions** through **RBAC**

Role	agent-runs	jobs	tools	tenants	Graph (write)
admin	CRUD	CRUD	CRUD	CRUD	✓
user	CR	CRD	R	×	×



Cineca Agent

Admin User Signed in as user

You 9:55:03 AM
What distinct relationship types exist from :Blast?

Agent 9:55:04 AM
Completed 39.9s

Execution Steps (3)

- ✓ Create TODO list 4ms
- ✓ graph.query 56ms
Success • 1 rows

```
<> Cypher Query
MATCH (b:Blast)-[r]->(target) RETURN
DISTINCT type(r) AS relationship_type
LIMIT $limit
```
- ✓ memgraph.response_builder 39.8s
The distinct relationship type(s) from :Blast is: 'OUTPUT'.

1/1 tasks completed

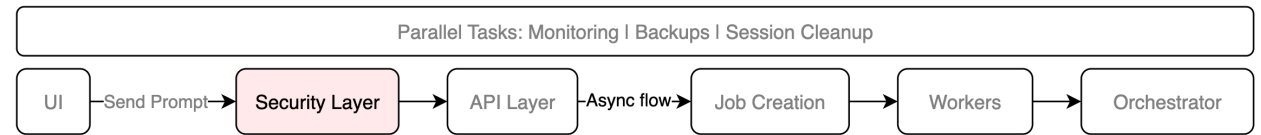
The distinct relationship type(s) from :Blast is: 'OUTPUT'.

phi3-mini

Type your message... (Enter to send)

SECURITY LAYER

8 safeguards before business logic



Request Protection Layers

1. CORS Policy

Trusted origin URLs only

2. Trace Context

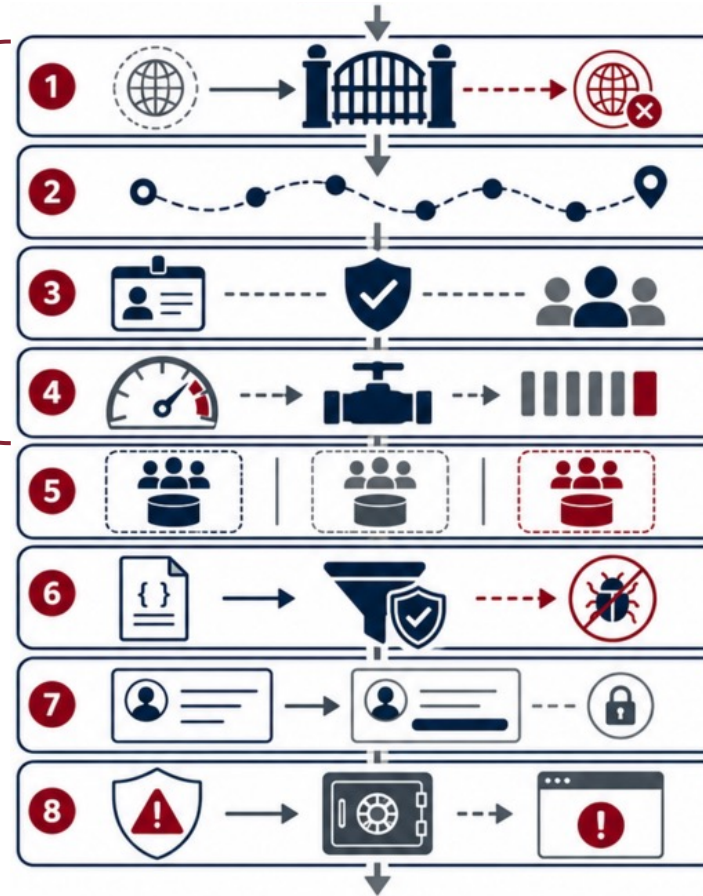
Adds trace ID for tracking

3. JWT + RBAC

Access control for Admin/User

4. Rate Limiting

Requests-per-minute control



5. Tenant Resolution

Organisation-level data isolation

6. Input Validation

Blocks unsafe requests and DB injection

7. Output Protection

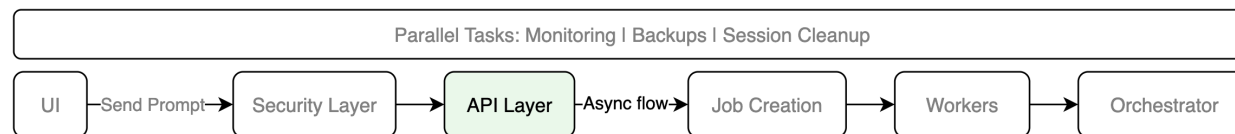
Sensitive data removal

8. Standard Error Handling

Standardized safe error responses

API LAYER - ROUTERS

76 endpoints



Core Platform APIs

- **Health** → Checking the health of the system
- **Authentication** → Validating JWT and roles of the users
- **Agents** → Managing agent settings
- **Tenants** → controlling tenant DB isolation and limits
- **Models** → Choosing LLM providers and models
- **Tools** → Invoking different MCP tools

Execution APIs

- **Jobs** → Creating and seeing async jobs and progress
- **Orchestrator** → Running Agent, Check status, and results
- **Graph** → Executing Cypher commands
- **Sessions** → managing conversation and runtime context

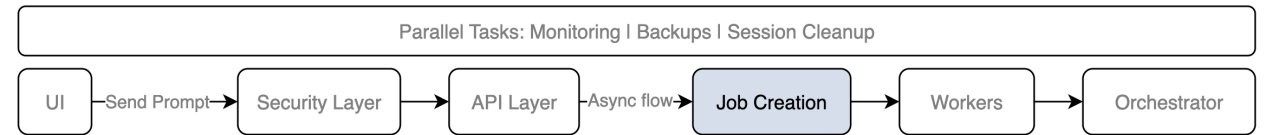
Operational APIs

- **Admin** → Monitoring system and stats
- **Export** → Exporting runs, job results, and logs



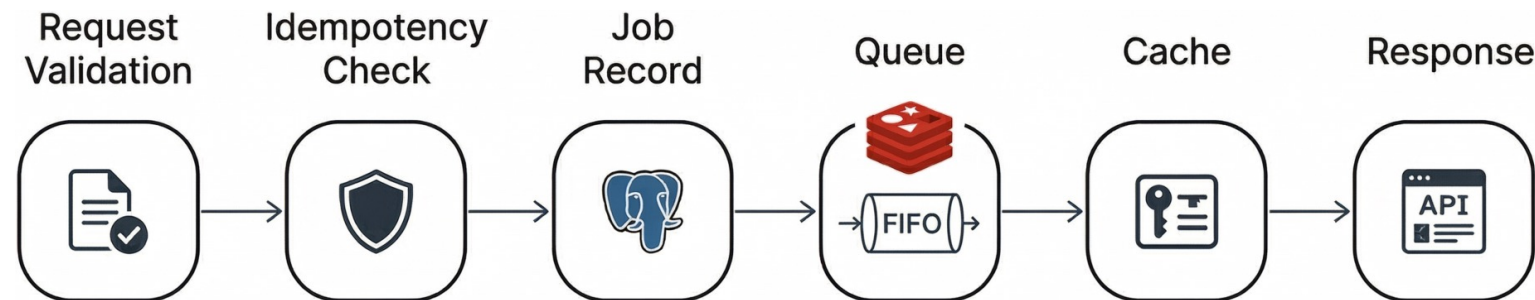
JOB CREATION

Long-running async task creation



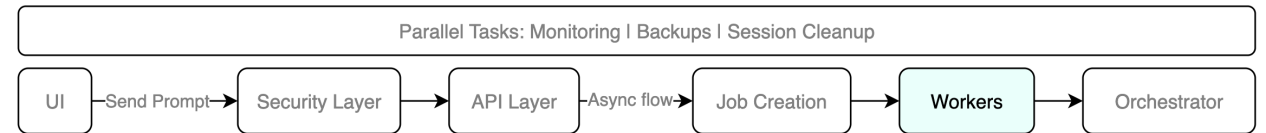
For each request we create an async job in a Redis queue:

1. **Request Validation** → Checks **required fields** and payload **structure**
2. **Idempotency Check** → Prevents **duplicate** jobs during retries
3. **Durable Job Record** → Stores the job in **PostgreSQL** with status queued
4. **Redis Queueing** → Pushes the job ID to **Redis queue** for worker execution
5. **Retry Cache** → **If job already existed**, returns the existing job ID
6. **Async Response** → Returns HTTP 202 for **Created** or **Accepted job ID** and current status



WORKERS

Job Processing Engine

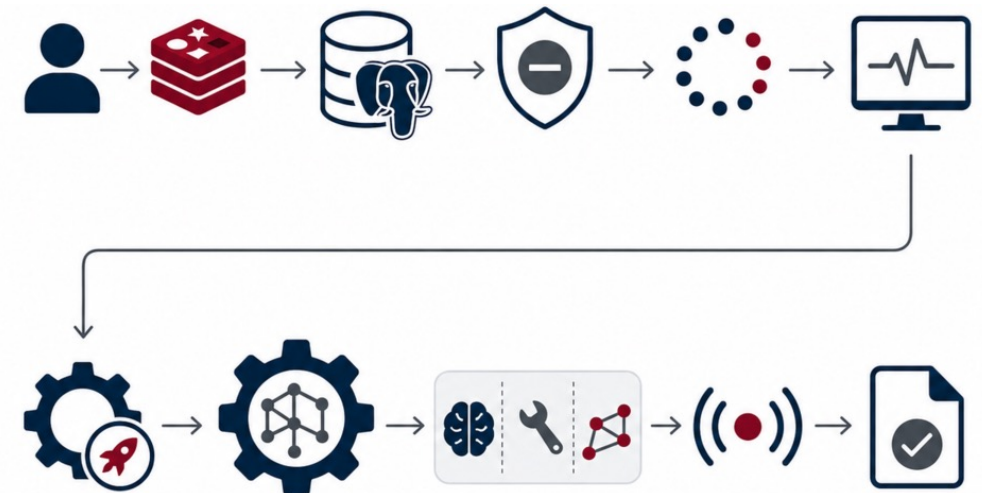


Worker Lifecycle

1. **Pull Job** → locks a queued job in Redis for one worker
2. **Load Job Data** → reads job details from PostgreSQL
3. **Cancel Check** → if user cancels, it stops safely
4. **Update Status** → records the job status
5. **Heartbeat** → Every 30s, detects stuck workers

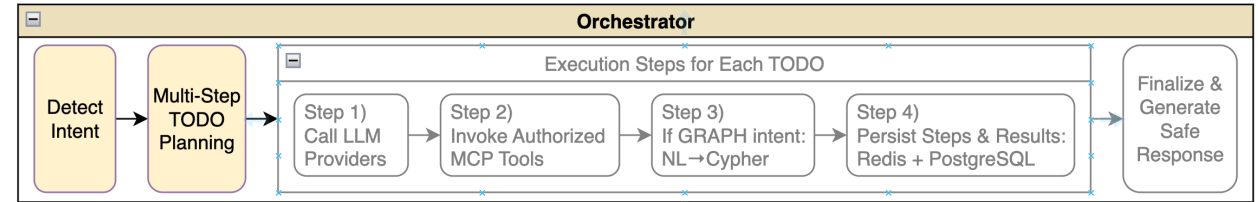
Agent Run Execution

6. **Init Agent Run** → Start the Agent for the job
7. **Orchestrator** → executes the workflow step by step
8. **Store Steps** → stores output of each step
9. **Live Progress** → sends live updates to the UI (SSE)
10. **Finalize Job** → applies guards, metrics, and final status



ORCHESTRATOR (Part 1/3)

Intent detection & Multi-step planning



Phase 1: Intent Classification

One LLM call classifies the request.

- **CHAT** → Conversational answer
- **GRAPH** → NL→Cypher query
- **SECURITY** → Access and permission request
- **ADMIN** → Administrative action
- **DANGEROUS** → Unsafe request, safely refused
- **EXPLAIN** → Analysis without execution



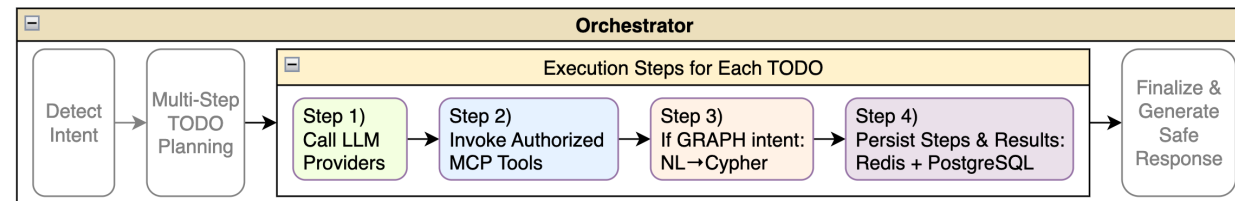
Phase 2: TODO Planning

The orchestrator turns the request into a clear **execution plan**.

- Breaks the request **into smaller actions**
- Uses **session context** and previous messages
- Selects **relevant** schema and **MCP tools**
- **Orders the actions** before execution
- Example: generate cypher → Run query in graph DB → analyze results → summarize answer

ORCHESTRATOR (Part 2/3)

Step Execution



Phase 3: Step-by-Step Execution of each TODO

1. Call the LLM Provider (Ollama / OpenAI)

Responsible for executing the process.

- **Retries** → retry failed LLM calls
- **Fallbacks** → switch provider/model if needed
- **Circuit breakers** → pause failing providers

2. Invoke the right MCP Tool for each TODO item

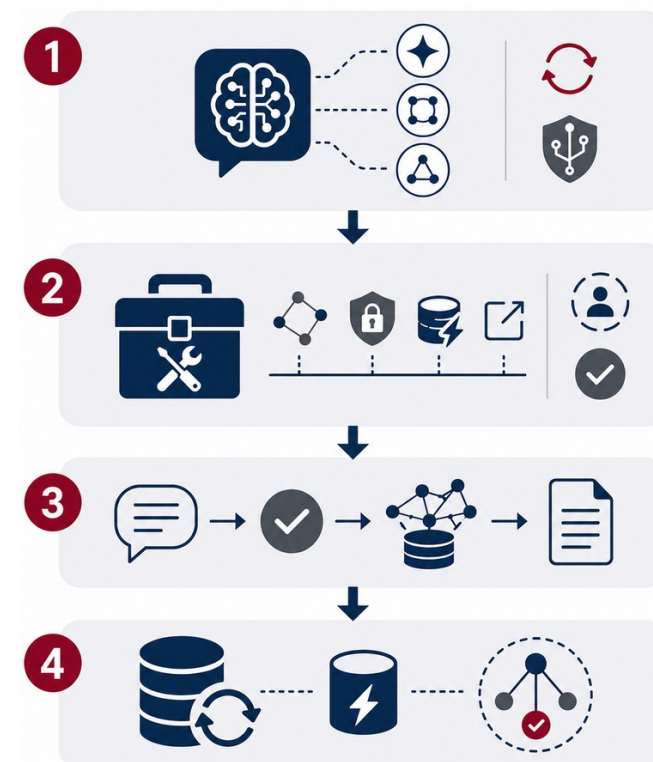
- **Tools:** graph, security, cache, export, system
- Before using each tool, role, tenant, and permissions are checked

3. Graph Pipeline (If intention is GRAPH)

- NL→Cypher generation, validation, execution, summarization

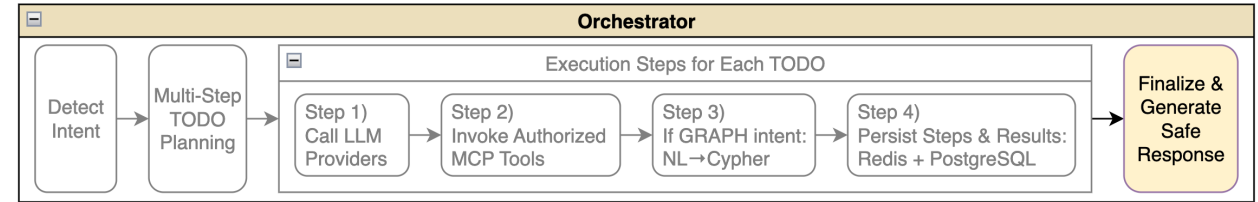
4. Persist Steps & Runtime State

- **PostgreSQL** → Tenants, jobs, runs, steps, inputs/outputs, timings, errors, final status, model configs
- **Redis** → Job queues, locks, heartbeats, cache sessions, progress events, cancellation flags
- **Memgraph** → Bioinformatics entities (Users, Institutions, Blast, Fasta) and their relationships.



ORCHESTRATOR (Part 3/3)

Finalization & Safe Response Generation



Phase 4: Final Output Pipeline

The orchestrator turns completed steps into a safe, consistent, and traceable final response.

1. **Assemble Step Outputs** → Combines step outputs into one final answer
2. **Fallback Handling** → Uses cached or default responses if generation fails
3. **Output Normalization** → Converts the result into readable text and optional structured JSON
4. **Output Safety Checks** → Removes sensitive data and filters unsafe content
5. **Final Persistence** → Saves final status, output, and metrics in PostgreSQL
6. **Observability Data** → Records metrics and traces for monitoring (Prometheus, Grafana)

```
{  
  "goal": "Find collaborating institutions",  
  "steps": ["query_graph", "analyze_results", "summarize_answer"],  
  "output": "Found 5 collaborating institutions.",  
  "metrics": { "duration_ms": 2340, "tokens": 1850 }  
}
```


VALIDATION & RESULTS

End-to-end verification

Validation Coverage

- **1,000+ automated tests** → for core logic, API routes, jobs, RBAC, tenancy, and orchestration
- **Security checks tested** → read-only enforcement, tenant isolation, RBAC, unsafe query blocking
- **Async execution tested** → job creation, streaming, cancellation, retries, and final status persistence

Full NL→Cypher path verified:

- Natural language → GRAPH intent → Cypher → validation → Memgraph Execution → final answer

Real Example:

User Request:

"Count Blast nodes grouped by presence of blast_version"

1. **Intent Classification** → GRAPH
2. **Cypher Generation** → read-only query created:
`MATCH (b:Blast) RETURN b.blast_version IS NOT NULL AS has_blast_version,
count(b) AS count ORDER BY has_blast_version DESC`
3. **Safety Validation** → ✓ Safety Validate → ✓ Read-only ✓ Tenant-safe ✓ RBAC ✓ Timeout-protected
4. **Memgraph Execution** → returns 1 grouped row
5. **Final Summary** → "In total, there are 39 Blast nodes, all containing blast_version attribute."

COMPARISON WITH RELATED FRAMEWORKS

Why existing frameworks were not enough

Platform	Full Stack	Agentic Planning	Governed Tools	Enterprise Security	Graph NL→Cypher	Observability	Summary
CAP	✓	✓	✓	✓	✓	✓	Complete platform for secure, observable graph-agent execution
Temporal	×	×	×	✓	×	✓	Durable workflow engine without native agent or graph capabilities
LangGraph	×	✓	~	×	~	~	Strong agent orchestration; APIs, security, and operations are external
OpenAI Agents SDK	×	✓	~	×	~	~	Fast agent development; platform governance must be added
Semantic Kernel	×	✓	~	~	~	~	Flexible AI orchestration; deployment is external
n8n	✓	~	~	~	×	~	Strong visual automation; limited for secure graph-agent workloads

Main takeaway:

CAP combines agent orchestration, durable jobs, governed tools, secure graph access, multi-tenancy, and observability into one platform.

✓ = Native support
~ = Partial / requires customization
× = Not supported

CONCLUSION + FUTURE WORK

From natural language to safe, traceable graph actions

Key Contribution

This thesis developed a multi-tenant AI platform that turns natural-language requests into safe, controlled, and traceable graph actions.

The platform connects:

- **Agents** for multi-step reasoning and execution
- **MCP tools** for controlled platform actions
- **Async jobs** for long-running tasks
- **Secure graph access** for validated NL→Cypher queries
- **Observability** for tracing, monitoring, and debugging

Future Work

- Add a lightweight Ask Mode for simple Q&A
 - Improve handling of unclear, mixed, or risky prompts
 - Recover interrupted workflows from the last completed step
 - Manage job queues with priorities, tenant limits, and overload protection
- Example: short chat jobs can run before long export jobs*

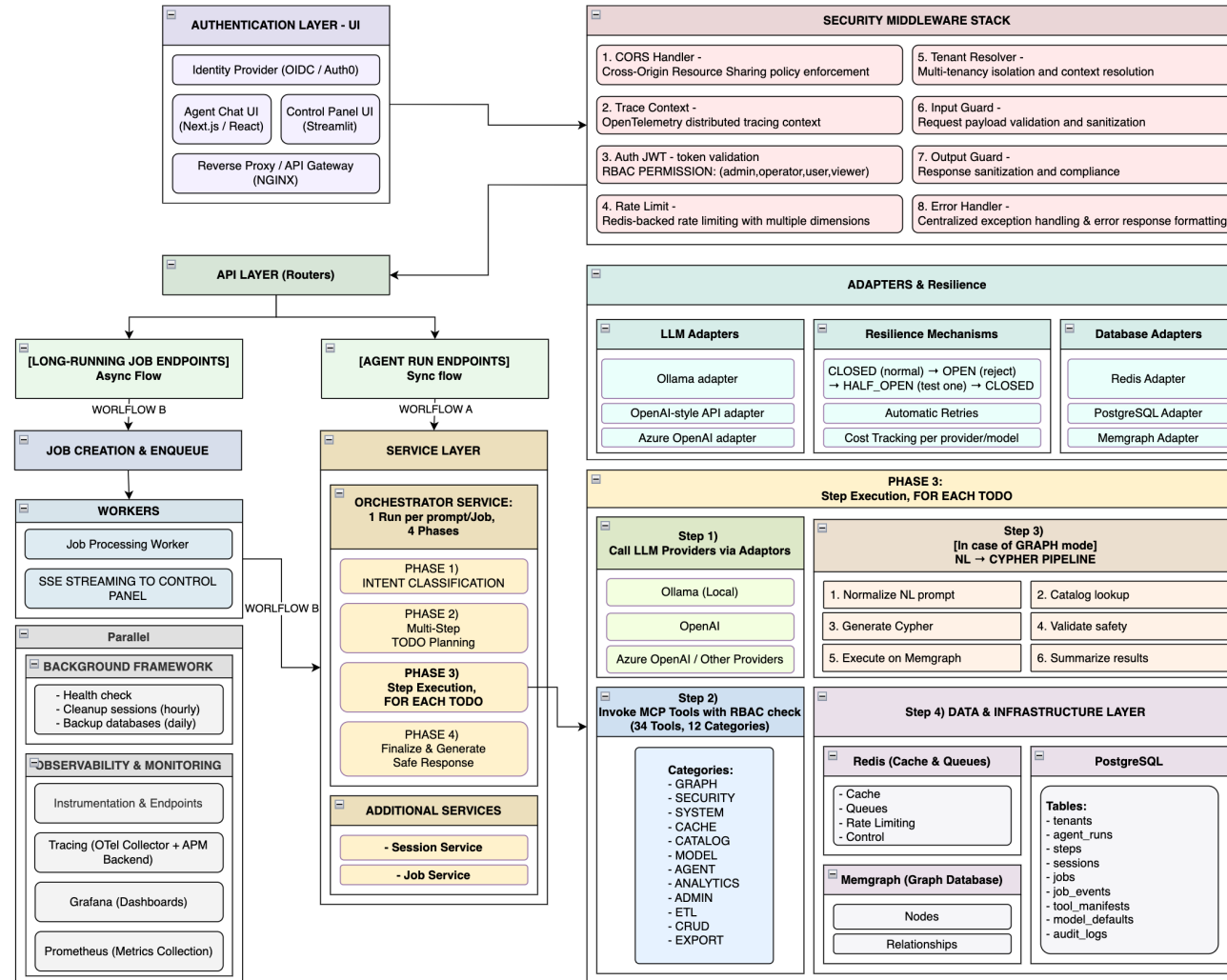


THANK YOU FOR YOUR ATTENTION

Feel free to ask any questions

BACKUP SLIDES

Backup Slides - Detailed System Architecture



LIMITATIONS

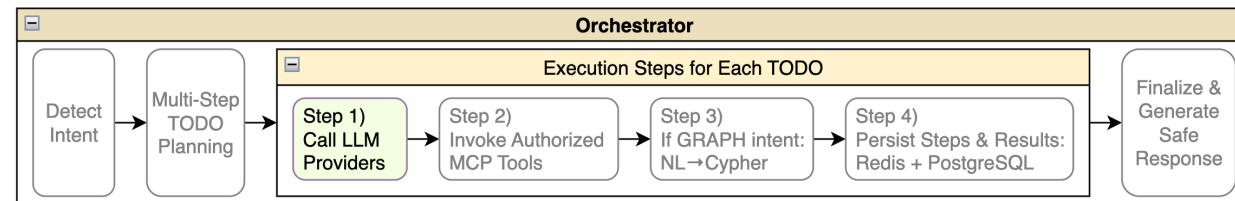
Evaluation constraints and future validation needs

- **HPC Availability** → CINECA/HPC evaluation was planned but unavailable
- **Limited Scenarios** → Testing used only a limited set of scenarios
- **Local Model Latency** → Powerful local models were too slow on a laptop
- **Long Runtime** → Multi-step runs could take over 20 minutes
- **Model Trade-off** → A lightweight model balanced speed and accuracy
- **Prompt Handling** → Unclear or risky prompts need better handling
- **Clarification Logic** → Clarification logic and safe defaults need refinement
- **LLM Variability** → Different models may generate different plans or queries
- **Residual Risk** → Validation reduces this risk, but cannot remove it fully



LLM PROVIDERS

Model Selection



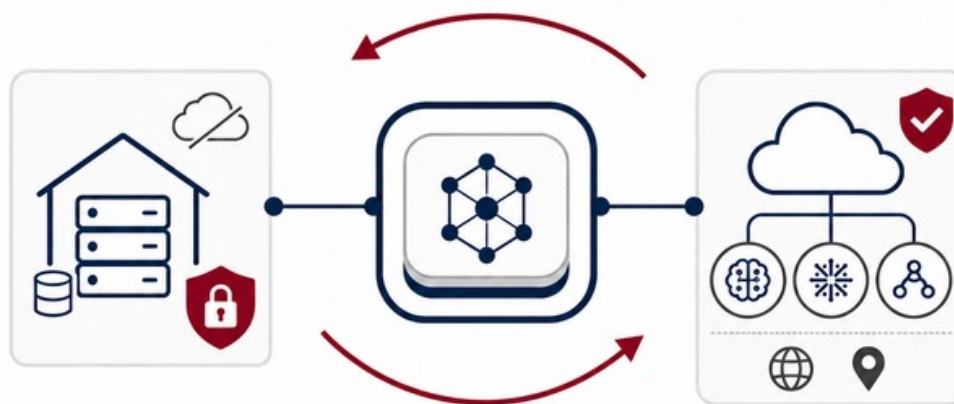
Provider & Deployment Options

- **Local** → Ollama-based execution for data control and lower external API costs
- **Cloud** → OpenAI / Azure OpenAI for managed, high-quality models
- **Flexible Logic** → switch providers without changing the orchestrator
- **Local Models** → Phi-3, Mistral, LLaMA, Qwen, Gemma models

Model Selection Order

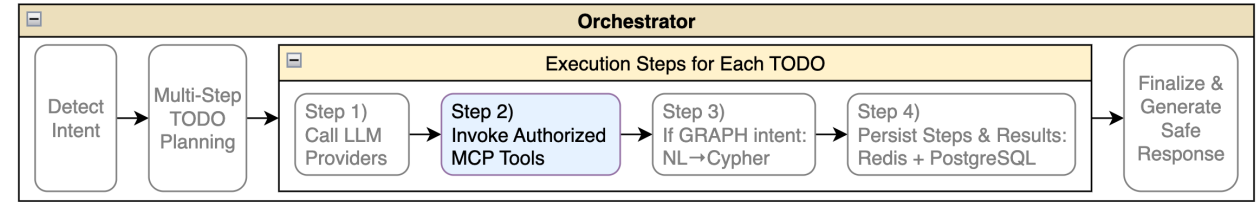
The platform uses the most specific model setting available, then falls back step by step to broader defaults.

Request → Session → Tenant → Global Default



MCP RUNTIME & TOOLS

34 tools across 12 categories



Runtime Controls

- **Registry** → stores tool definitions, schemas, scopes, and limits
- **Access** → checks RBAC, tenant context, and tool permissions
- **Validation** → verifies inputs before execution
- **Audit** → records tool calls, results, errors, and timing
- **Output** → returns structured outputs to the agent

Invocation Flow

Request → Validate → Authorize → Execute → Audit → Return

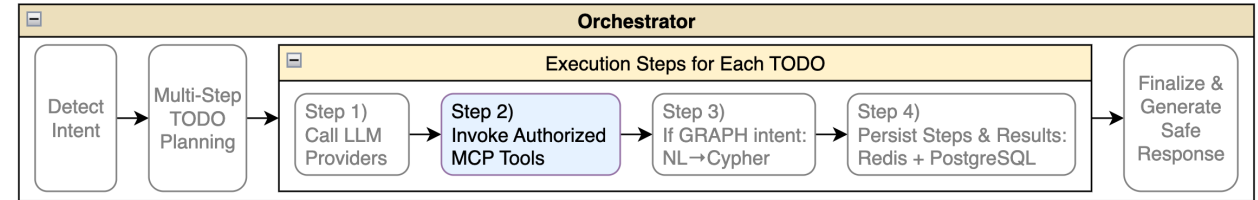
Main Tool Families

- **Graph** → Memgraph schema, Cypher execution, graph queries
- **Security** → identity, permissions, and access checks
- **System & Catalog** → health, metrics, configuration, and tool discovery
- **Model & Agent** → model operations, context, history, and session state
- **Analytics** → summarizes graph results and prepares data for charts
- **Admin / ETL / CRUD / Export** → restricted operations, data import, update, and export



MCP RUNTIME & TOOLS

34 tools across 12 categories



MCP

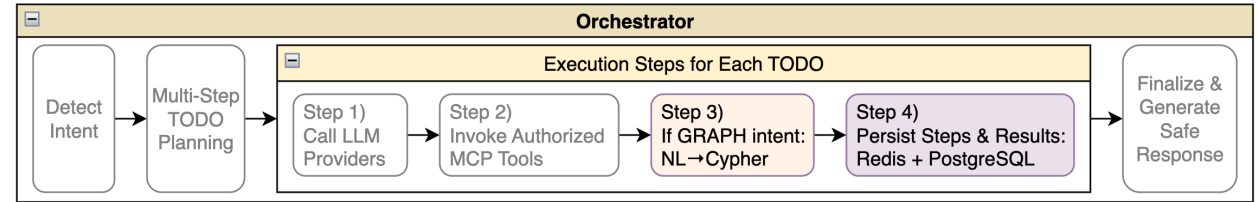
- MCP provides a permission-aware and traceable execution layer between the agent and the platform.
- Every MCP tool is registered, validated, authorized, and audited before execution.

Categories

- **Graph** → Query Memgraph, generate Cypher, inspect schema, and explain query plans.
- **Security** → Check identity, roles, scopes, permissions, and restricted operations.
- **System** → Monitor platform health, metrics, configuration, workers, and queues.
- **Cache** → Read, store, delete, and invalidate cached Redis results.
- **Catalog** → Discover tools, resources, graph entities, and available platform metadata.
- **Model** → List, inspect, warm up, and switch between available LLM providers.
- **Agent** → Manage context, interaction history, and session-level state.
- **Analytics** → Run analytical queries, aggregations, ratios, and chart-ready outputs.
- **Admin** → Manage indexes, constraints, and admin-only database operations.
- **ETL** → Import, export, and transform graph data in bulk.
- **CRUD** → Create, update, and delete graph nodes with permission checks.
- **Export** → Convert query results into CSV, JSON, or Cypher formats.

DATA LAYER

Redis, PostgreSQL, Memgraph



Redis → Fast Runtime State

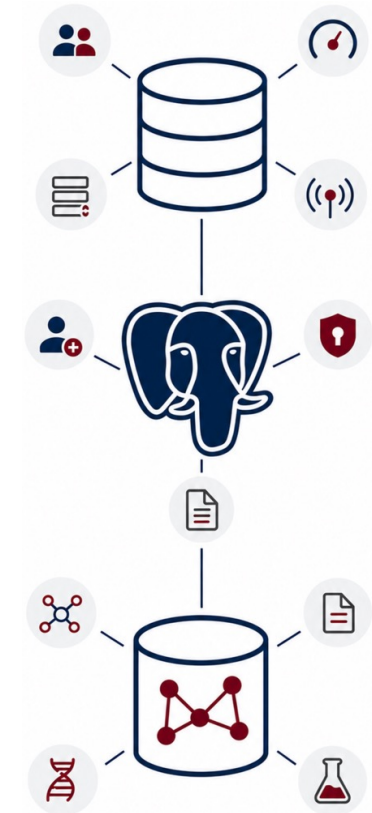
- Caches sessions, tenant settings, and model defaults
- Manages rate limits, job queues, and progress events
- Stores cancellation flags, idempotency keys, and circuit-breaker state

PostgreSQL → Durable Platform State

- Stores tenants, sessions, runs, steps, and jobs
- Records job events, tool calls, audit logs, and encrypted secrets
- Manages model/provider configuration and idempotency records

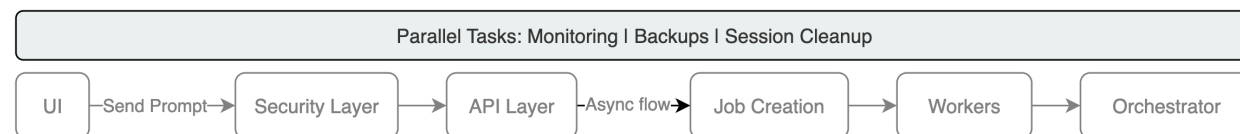
Memgraph → Graph Domain Data

- Stores users, institutions, commands, files, and bioinformatics artifacts
- Represents BLAST analyses, sequences, and FASTA/XML files
- Connects entities through **WORKS_AT**, **RUNS**, **INPUT**, and **OUTPUT** relationships



BACKGROUND TASKS

Monitoring | Backups | Session Cleanup



Scheduled APScheduler Jobs

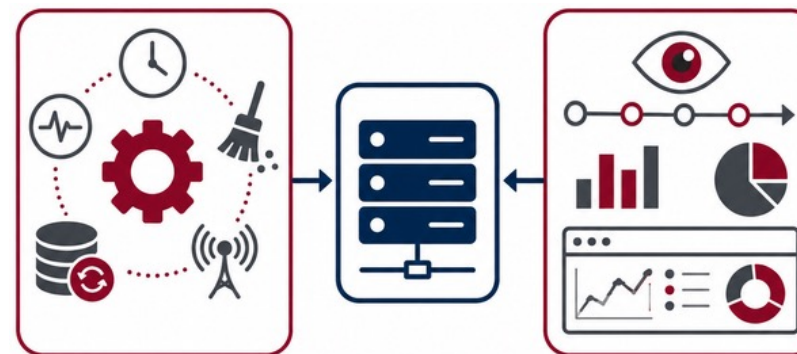
- **Every 30s** → **Health Check** → checks PostgreSQL, Redis, Memgraph, and LLM providers
- **Every 60s** → **Provider Monitoring** → tracks LLM health, metrics, and circuit breakers
- **Hourly** → **Cleanup** → removes expired sessions, cache, old jobs, and orphan runs
- **Daily** → **Backup** → saves PostgreSQL, Redis, Memgraph, and audit snapshots

Observability Stack

- **OpenTelemetry** → records request traces and shows time spent in LLM calls, DB queries, tools, and jobs
- **Prometheus** → stores metrics for latency, errors, queues, tokens, and provider health over time
- **Grafana** → visualizes metrics and traces for monitoring, debugging, and cost tracking

Operational Value

- Detects failures and provider issues early
- Finds slow requests and failed tools
- Tracks queues, costs, recovery, and debugging



ADAPTERS

Adapters let the platform switch providers or databases without changing the orchestrator.

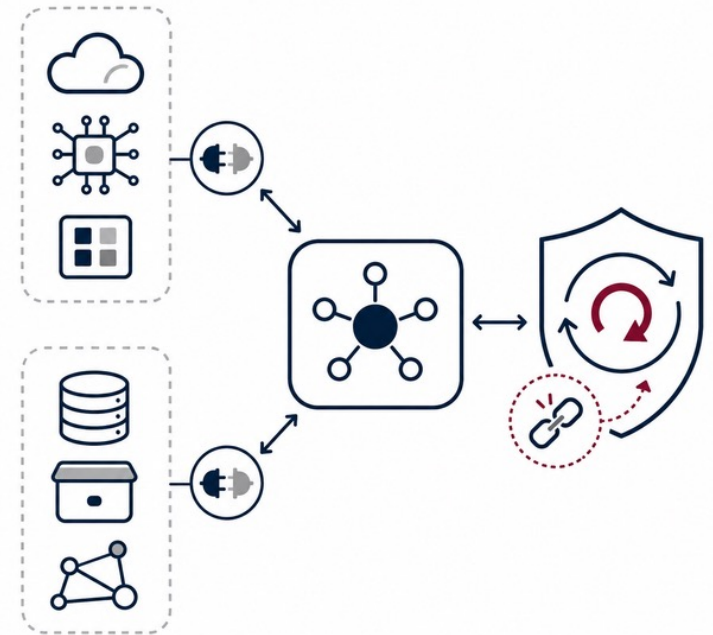
Swappable Adapters

Adapters let the platform switch providers or databases without changing the orchestrator.

- **LLM adapters** → Ollama, OpenAI, Azure OpenAI
- **Provider independence** → business logic stays the same
- **Database adapters** → Redis, PostgreSQL, and Memgraph
- **Backend flexibility** → The orchestrator remains unchanged

Robustness Mechanisms

- **Retries with backoff** → retry failed calls with increasing delay
- **Provider fallback** → switch to another provider if one fails
- **Circuit breakers** → pause unstable providers
- **Fail-fast behavior** → skip broken services quickly
- **Recovery flow** → [CLOSED → OPEN → HALF_OPEN → CLOSED]



API - ENDPOINTS IN DETAIL

Main endpoint groups and responsibilities

Endpoint	Description
/v1/health	Checks if the platform is running, ready, and fully started. Reports PostgreSQL, Redis, and Memgraph status.
/v1/auth	Verifies user identity and validates OIDC/JWT authentication tokens.
/v1/agents	Manages agent definitions, prompt templates, and behavior settings.
/v1/tenants	Manages tenant organizations, default models, rate limits, quotas, and enabled features.
/v1/models	Registers LLM providers, configures models, API keys, and tenant defaults.
/v1/tools	Discovers MCP tools, reads tool schemas, and invokes tools with validated inputs.
/v1/jobs	Creates long-running async jobs, streams progress, and cancels running jobs.
/v1/agent-runs	Starts, tracks, retrieves, and cancels agent execution runs.
/v1/graph	Executes direct Cypher queries and validated NL→Cypher graph queries.
/v1/sessions	Manages chat sessions, conversation history, and runtime context.
/v1/batch	Handles bulk requests, such as multiple agent runs or tool calls in one request.
/v1/admin	Runs restricted admin actions such as cache clearing, maintenance, migrations, and system stats.
/v1/internal	Provides restricted diagnostics, configuration checks, and internal debugging tools.
/v1/export	Exports agent runs, job results, and audit logs in CSV or JSON format.